Table of Contents

PART A: The Data Story – England's Fertility Journey (2010–2023) 2

Introduction 2

The Big Question: Are national averages a true reflection of local life? 2

A Country in Decline 3

Visualisation 1: National vs. Regional Trends (The Big Picture) 3

Visualisation 2: Percent Change (The Speed of the Drop) 4

The Regional Fingerprint 5

Visualisation 3: Regional Year-by-Year Comparison (The Heatmap) 5

Visualisation 4: Side-by-Side: Local Trajectories vs. National Average 6

The "Masking" Effect 7

Visualisation 5: Local Variation 7

Visualisation 6: The Growing Gap (Highest vs. Lowest Local Districts) 8

Conclusion 9

PART B: Critical Reflection 10

Methodology 10

Data selection 10

Tools 10

Exploratory Data Analysis 15

How the research question was refined 21

Use of Generative AI 22

PART A: The Data Story – England’s Fertility Journey (2010–2023)

Introduction

The Big Question: Are national averages a true reflection of local life?

Every year, the news reports on the "National Fertility Rate," giving us one single number for the whole of England. In 2023, that number was nearly 49 births for every 1,000 women (Refer to Figure 22 under methodology for values). My goal is to see if the national average we see in the news actually represents what is happening in local neighbourhoods. I want to find out if looking at statistics of the whole country, “masks” bigger divergences in numbers within smaller regions.

A Country in Decline

Visualisation 1: National vs. Regional Trends (The Big Picture)

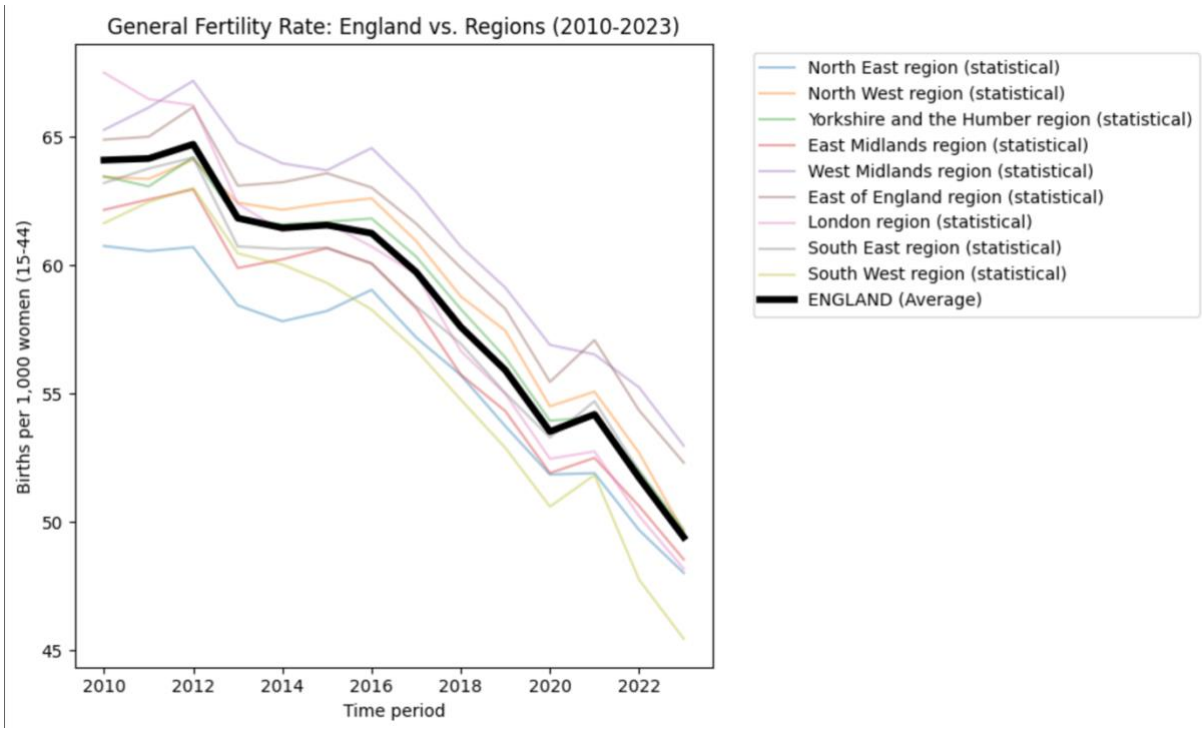

Figure 1: General Fertility Rate: England vs. Regions (2010-2023)

The first thing we notice when looking at the data from 2010 to 2023 is a downward trend. In 2010, the General Fertility Rate (GFR) was 64.1. By 2023, it dropped to 49.4. (Refer to Figure 22 under methodology for values). When we look at the nine statistical regions of England, they all follow the same national trend. Whether it is the North East or the South West, direction is same: its down.

Visualisation 2: Percent Change (The Speed of the Drop)

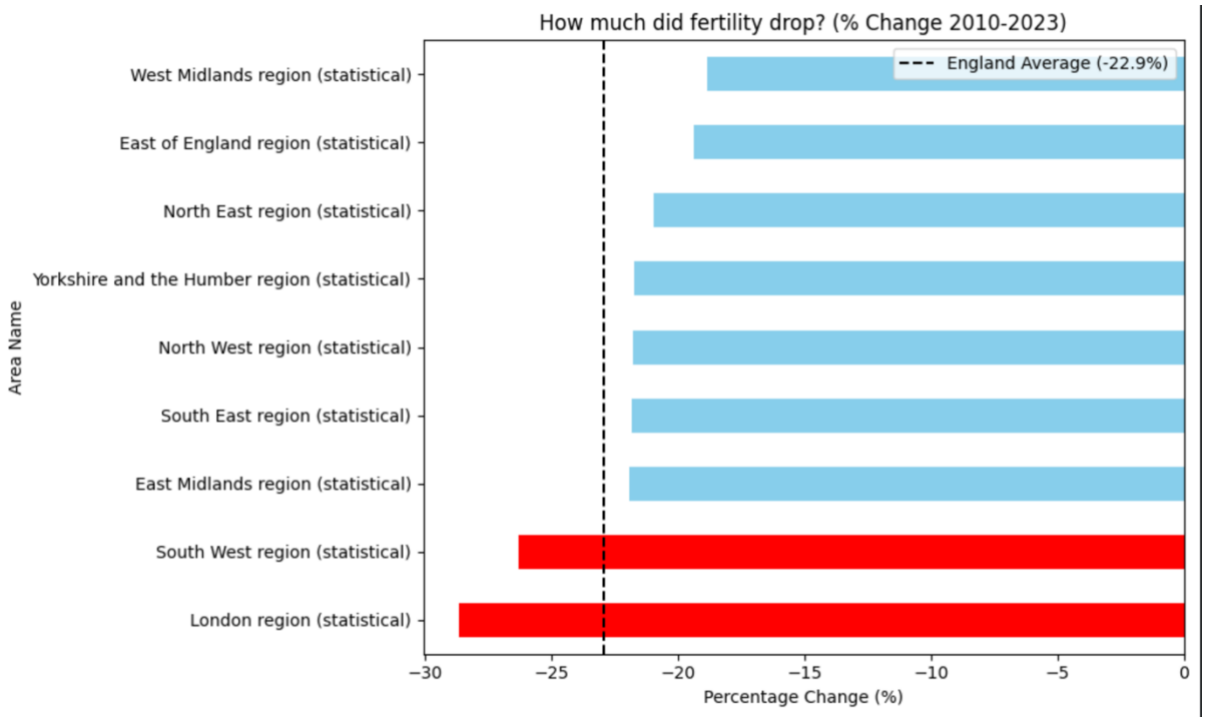

Figure 2: A percentage-change analysis from 2010 to 2023.

Although the nine statistical regions of England, all follow this national "heartbeat trend”, the speed of this GFR drop varies. The above visualization summarizes the rates (speed) of this drop. London and the South West region have seen the most aggressive GFR crashing rates. The rate of GFR drop for London and the South West region is even higher than the England average 22.9%. This further strengthens that although the reasons for having fewer children are nationwide, they are hitting some regions harder than others.

The Regional Fingerprint

Visualisation 3: Regional Year-by-Year Comparison (The Heatmap)

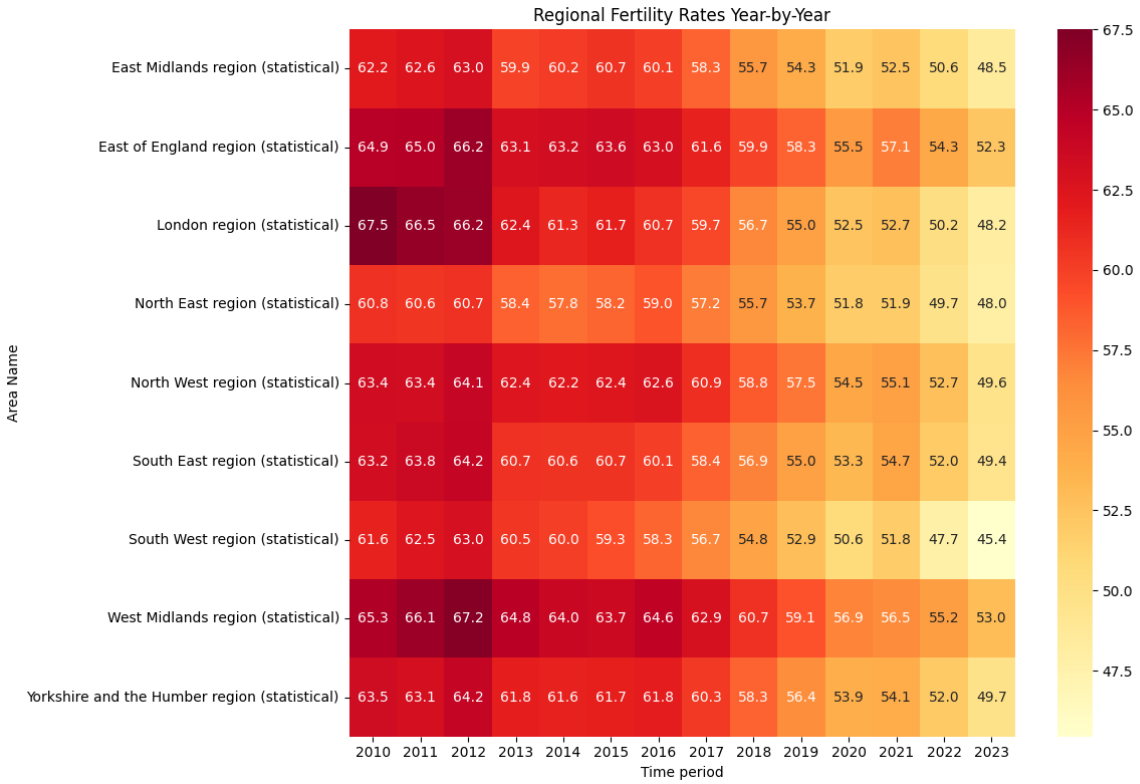

Figure 3: Heatmap of regional fertility rates year by year

When you look at the map left to right, you can see the colors "cooling down." But, even though the whole map is fading, notice that colors don't mix into one messy blur. You can still see distinct horizontal stripes. For example, West Midlands row usually stays a darker shade than South West row every single year. This is the "Regional Fingerprint." Each region has its own uniqueness in GFR changes meaning that while the whole country is cooling down, each region cools in its own pase. This suggests that GFR isn't just about national factors like national cost of living. It is tied to deep, local factors maybe like types of jobs available in a region, cost of local housing and stubborn cultural practices that don’t easily change.

Visualisation 4: Side-by-Side: Local Trajectories vs. National Average

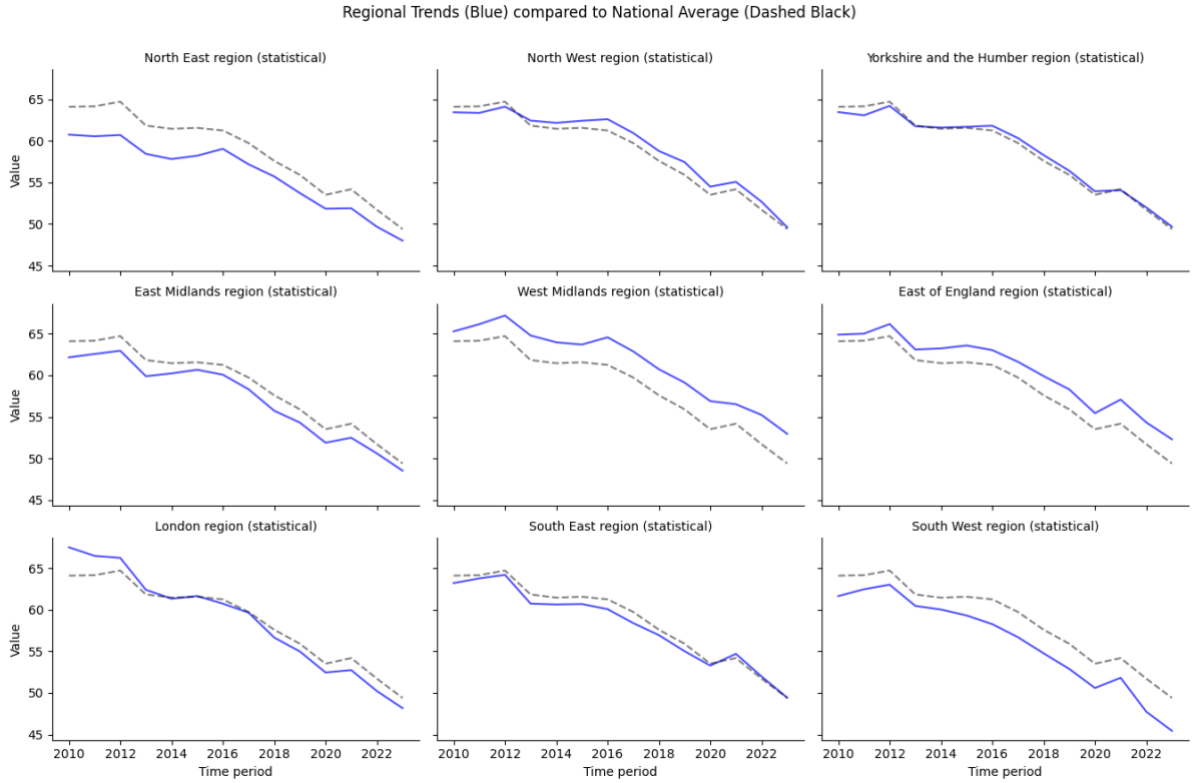

Figure 4: : Regional Trends compared to National Average

When we look at these regions side-by-side against the national average, we see that most regions stay on their "side" of the line for the entire 13-year period. For example, West Midlands never drops below the national line. South West never rises above it. This further strengthens the "Regional Fingerprint". It shows although the national trend is real, but it sits on top of regional differences and uniqueness that are very stubborn and slow to change. This also proves that a "national average" is just a mathematical middle point that no region actually experience.

The "Masking" Effect

The biggest discovery in this data comes when we move from large statistical regions to even smaller areas; the local Districts. This is where the national average finally fails to tell the truth.

Visualisation 5: Local Variation

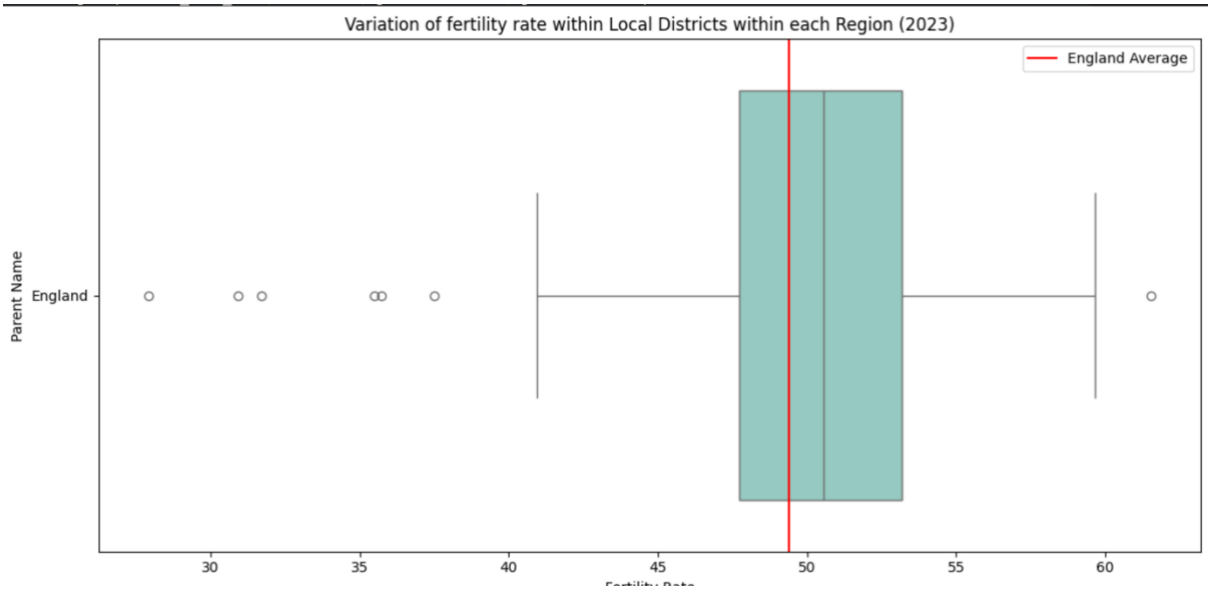

Figure 5: Variation in Local Areas (Districts) within each Region (2023)

While the national average is around 48, some local areas have rates near 28, while some others are above 60. By only looking at the "England Average," we totally ignore these extreme differences. A national policy designed for a rate of 48 would be useless for a town struggling with rate of 28.

Visualisation 6: The Growing Gap (Highest vs. Lowest Local Districts)

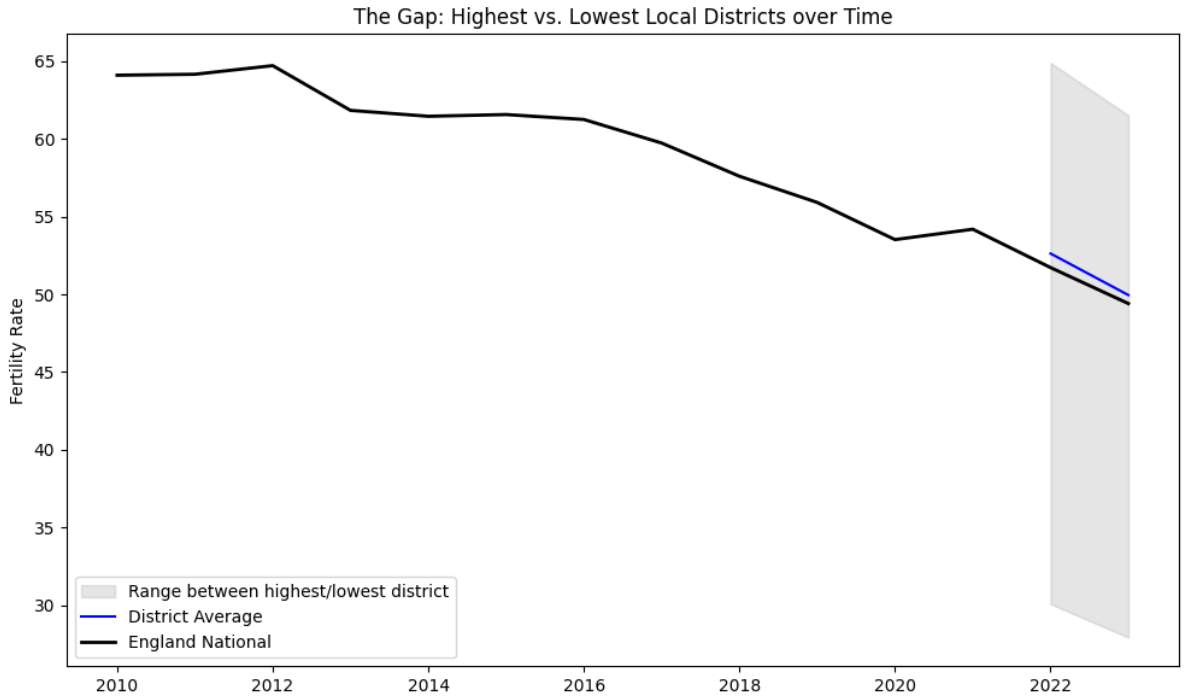

Figure 6: The Gap: Highest vs. Lowest Local Districts over Time

This chart tracks the gap between the highest-fertility district and the lowest-fertility district in England. As you can see, the gap between the top and the bottom remains massive (the grey shaded area). Spatial inequality is not disappearing; this variation is being "masked" by only looking at the national average (black line).

Conclusion

So, do national trends genuinely represent what is happening everywhere? Only partially. The national average accurately represents the direction, but it fails to represent the local reality.

The smaller the geographic scale we look at, the more inequality we find. When we go from national level to statistical regions to districts, the divergence from the national average is getting bigger. To truly understand fertility in England and make policies and decisions regarding that, we must look past the national blanket and see the underlying patch of local stories.

PART B: Critical Reflection

Methodology

Data selection

Data was obtained from https://fingertips.phe.org.uk.

Indicator ID: 92266

Indicator name: General fertility rate.

Tools

Google Collab; a professional data analysis environment is used for preprocessing and EDA. Python libraries Matplotlib and Seaborn were utilized for visualization as they allow a wide range of customization.

Preparation

After loading dataset, columns were analysed, values were set to numeric and unwanted columns and rows were removed.

```
##IMPORTING DATA SET AND PREPROCESSING

#mounting drive

from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
#importing data set

import pandas as pd
df= pd.read_csv("/content/drive/MyDrive/Data Viz/Raw Data/Indicator-92266-DATA.csv")
df.head()
```

	Indicator ID	Indicator Name	Parent Code	Parent Name	Area Code	Area Name	Area Type	Sex	Age	Category Type	...	Count	Denominator	Value note	Recent Trend	Compared to England value or percentiles
0	92266	General fertility rate	NaN	NaN	E92000001	England	England	Female	15-44 yrs	NaN	...	687007.0	10718101.0	NaN	NaN	Not compared
1	92266	General fertility rate	E92000001	England	E06000001	Hartlepool	UA	Female	15-44 yrs	NaN	...	1134.0	17895.0	NaN	NaN	Similar
2	92266	General fertility rate	E92000001	England	E06000002	Middlesbrough	UA	Female	15-44 yrs	NaN	...	2138.0	28283.0	NaN	NaN	Higher
3	92266	General fertility rate	E92000001	England	E06000003	Redcar and Cleveland	UA	Female	15-44 yrs	NaN	...	1585.0	25185.0	NaN	NaN	Similar

Figure 7: Importing data set

```python
#no. of rows and columns

print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```python
#analysing columns

column_names = df.columns
print(column_names)
```

```
Index(['Indicator ID', 'Indicator Name', 'Parent Code', 'Parent Name',
       'Area Code', 'Area Name', 'Area Type', 'Sex', 'Age', 'Category Type',
       'Category', 'Time period', 'Value', 'Lower CI 95.0 limit',
       'Upper CI 95.0 limit', 'Lower CI 99.8 limit', 'Upper CI 99.8 limit',
       'Count', 'Denominator', 'Value note', 'Recent Trend',
       'Compared to England value or percentiles', 'Column not used',
       'Time period Sortable', 'New data', 'Compared to goal',
       'Time period range'],
      dtype='object')
```

Figure 8: Analyze structure of dataset

```python
#removing unwanted columns


columns_needed = [
    'Area Code',
    'Area Name',
    'Area Type',
    'Parent Name', # Added 'Parent Name' here
    'Category',
    'Time period',
    'Value',  # Re-adding the 'Value' column
    'Count',
    'Denominator',
    'Time period range'
]

df_selected = df[columns_needed]

print(df_selected.head())
```

```
   Area Code             Area Name Area Type Parent Name Category  \
0  E92000001               England   England         NaN      NaN
1  E06000001             Hartlepool        UA     England      NaN
2  E06000002          Middlesbrough        UA     England      NaN
3  E06000003  Redcar and Cleveland        UA     England      NaN
4  E06000004       Stockton-on-Tees        UA     England      NaN

   Time period      Value     Count  Denominator Time period range
0         2010  64.097829  687007.0   10718101.0                1y
1         2010  63.369656    1134.0      17895.0                1y
2         2010  75.593112    2138.0      28283.0                1y
3         2010  62.934286    1585.0      25185.0                1y
4         2010  65.051322    2478.0      38093.0                1y
```

Figure 9: Removing unwanted columns

```python
# Ensuring values are numeric

df_selected = df_selected[df_selected['Category Type'].isna()].copy()
df_selected['Value'] = pd.to_numeric(df_selected['Value'], errors='coerce')
df_selected['Time period'] = pd.to_numeric(df_selected['Time period'], errors='coerce')
```

Figure 10: Ensuring the column values are numeric

```python
#Analysing the rows

df_selected['Area Type'].value_counts()
```

Area Type	count
UA	1844
District	328
County	300
CCG	214
Regions (statistical)	126
ICB sub-locations	67
Combined authorities	20
England	14
PHE Centre	9

dtype: int64

Figure 11: Analyzing the column "Area type"

For Area type= England, Regions (statistical) and District, separate data frame is created namely; “df_engalnd”, “df_Regions”, “df_districts”. 164 England records is due to irrelevant extra categorization; unwanted rows were removed.

```python
#Creating the England (National) data set
# Filter rows where Area Type is 'England'
df_england = df_selected[df_selected['Area Type'] == 'England']

# View the filtered rows
print(df_england)
```

```
      Area Code Area Name Area Type Parent Name  \
0     E92000001   England   England         NaN
455   E92000001   England   England         NaN
456   E92000001   England   England         NaN
457   E92000001   England   England         NaN
458   E92000001   England   England         NaN
...         ...       ...       ...         ...
2731  E92000001   England   England         NaN
2732  E92000001   England   England         NaN
2733  E92000001   England   England         NaN
2734  E92000001   England   England         NaN
2735  E92000001   England   England         NaN

                                     Category  Time period      Value  \
0                                         NaN         2010  64.097829
455                                       NaN         2011  64.157960
456             Most deprived decile (IMD2019)         2011  68.426930
457      Second most deprived decile (IMD2019)         2011  67.486260
458       Third more deprived decile (IMD2019)         2011  61.935480
...                                        ...          ...        ...
2731      Fifth less deprived decile (IMD2019)         2023  49.177070
2732     Fourth less deprived decile (IMD2019)         2023  49.394290
2733      Third less deprived decile (IMD2019)         2023  49.300480
2734   Second least deprived decile (IMD2019)         2023  48.653240
2735          Least deprived decile (IMD2019)         2023  50.717970
```

Figure 12: Creating new data set for national data

```python
# Keep only rows where Category is NaN (these are the national sum rows)
df_england_sum = df_england[df_england['Category'].isna()]
print(df_england_sum)
```

```
      Area Code Area Name Area Type Parent Name Category  Time period  \
0     E92000001   England   England         NaN      NaN         2010
455   E92000001   England   England         NaN      NaN         2011
628   E92000001   England   England         NaN      NaN         2012
801   E92000001   England   England         NaN      NaN         2013
974   E92000001   England   England         NaN      NaN         2014
1147  E92000001   England   England         NaN      NaN         2015
1320  E92000001   England   England         NaN      NaN         2016
1493  E92000001   England   England         NaN      NaN         2017
1666  E92000001   England   England         NaN      NaN         2018
1839  E92000001   England   England         NaN      NaN         2019
2012  E92000001   England   England         NaN      NaN         2020
2185  E92000001   England   England         NaN      NaN         2021
2358  E92000001   England   England         NaN      NaN         2022
2715  E92000001   England   England         NaN      NaN         2023

          Value     Count  Denominator Time period range
0     64.097829  687007.0   10718101.0                1y
455   64.157960  688118.0   10725373.0                1y
628   64.714150  694242.0   10727823.0                1y
801   61.835270  664517.0   10746569.0                1y
974   61.455160  661501.0   10763962.0                1y
1147  61.566040  664399.0   10791647.0                1y
1320  61.249710  663155.0   10827071.0                1y
1493  59.733080  646794.0   10828070.0                1y
1666  57.597260  625651.0   10862513.0                1y
1839  55.911310  610505.0   10919169.0                1y
2012  53.520350  585195.0   10934065.0                1y
2185  54.188300  595948.0   10997725.0                1y
2358  51.719990  577046.0   11157117.0                1y
2715  49.405980  563561.0   11406736.0                1y
```

Figure 13: Removing unwanted data rows in national data set

```python
#Creating the Regions (statistical) data set

# Filter rows where Area Type is 'Regions (statistical)'
df_Regions = df_selected[df_selected['Area Type'] == 'Regions (statistical)']

# View the filtered rows
print(df_Regions)
```

```
      Area Code                                     Area Name  \
156   E12000001                 North East region (statistical)
157   E12000002                 North West region (statistical)
158   E12000003  Yorkshire and the Humber region (statistical)
159   E12000004              East Midlands region (statistical)
160   E12000005              West Midlands region (statistical)
...         ...                                            ...
3057  E12000005              West Midlands region (statistical)
3058  E12000006            East of England region (statistical)
3059  E12000007                    London region (statistical)
3060  E12000008                 South East region (statistical)
3061  E12000009                 South West region (statistical)

                   Area Type Parent Name Category  Time period      Value  \
156   Regions (statistical)     England      NaN         2010  60.750463
157   Regions (statistical)     England      NaN         2010  63.439377
158   Regions (statistical)     England      NaN         2010  63.463875
159   Regions (statistical)     England      NaN         2010  62.160748
160   Regions (statistical)     England      NaN         2010  65.278395
...                     ...         ...      ...          ...        ...
3057  Regions (statistical)     England      NaN         2023  52.968760
3058  Regions (statistical)     England      NaN         2023  52.310510
3059  Regions (statistical)     England      NaN         2023  48.174690
3060  Regions (statistical)     England      NaN         2023  49.426490
3061  Regions (statistical)     England      NaN         2023  45.446160
```

Figure 14: Creating new data set for regional statistical data

```python
#Creating the Districts data set

df_districts = df_selected[df_selected['Area Type'] == 'District']

print(df_districts)
df_districts.shape[0]
```

```
      Area Code             Area Name Area Type Parent Name Category  \
2442  E07000008             Cambridge  District     England      NaN
2443  E07000009   East Cambridgeshire  District     England      NaN
2444  E07000010               Fenland  District     England      NaN
2445  E07000011       Huntingdonshire  District     England      NaN
2446  E07000012  South Cambridgeshire  District     England      NaN
...         ...                   ...       ...         ...      ...
2958  E07000241       Welwyn Hatfield  District     England      NaN
2959  E07000242    East Hertfordshire  District     England      NaN
2960  E07000243             Stevenage  District     England      NaN
2961  E07000244          East Suffolk  District     England      NaN
2962  E07000245          West Suffolk  District     England      NaN

      Time period     Value   Count  Denominator Time period range
2442         2022  30.07022  1229.0      40871.0                1y
2443         2022  55.13086   851.0      15436.0                1y
2444         2022  61.59797  1067.0      17322.0                1y
2445         2022  52.52631  1707.0      32498.0                1y
2446         2022  53.02639  1541.0      29061.0                1y
...           ...       ...     ...          ...               ...
2958         2023  45.69019  1181.0      25848.0                1y
2959         2023  53.67871  1519.0      28298.0                1y
2960         2023  51.38768   948.0      18448.0                1y
2961         2023  48.03913  1748.0      36387.0                1y
2962         2023  53.61021  1802.0      33613.0                1y

[328 rows x 10 columns]
328
```

Figure 15: Creating new data set for local district data

Exploratory Data Analysis

Visualization 1: Line Chart of General Fertility Rate- England vs. Regions (2010-2023)

```
##EXPLORATORY DATA ANALYSIS

import matplotlib.pyplot as plt
import seaborn as sns

#Visualisation 1:

plt.figure(figsize=(10, 6))
sns.lineplot(data=df_Regions, x='Time period', y='Value', hue='Area Name', alpha=0.4)
sns.lineplot(data=df_england_sum, x='Time period', y='Value', color='black', linewidth=4, label='ENGLAND (Average)')
plt.title('General Fertility Rate: England vs. Regions (2010-2023)')
plt.ylabel('Births per 1,000 women (15-44)')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Data Viz/viz1_national_vs_regional.png')
```

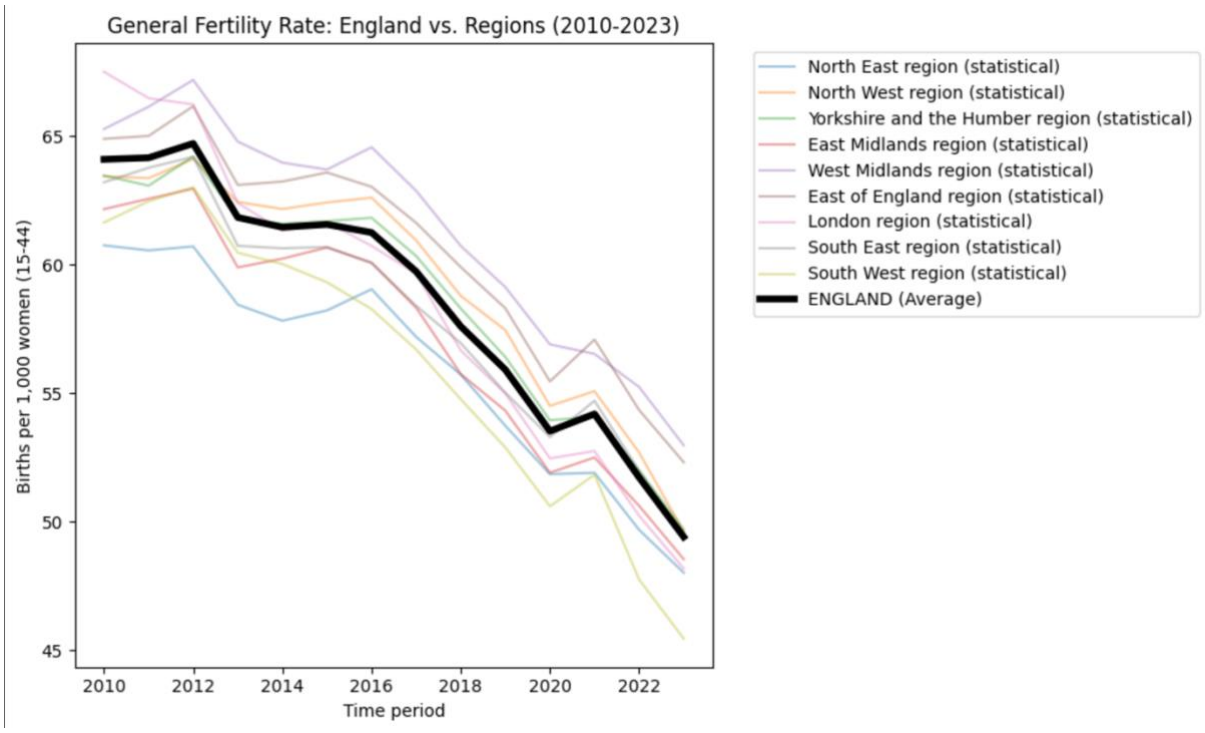

Figure 16: Visualization 1

This was drawn to establish the "Big Picture" to see how the GFR changes with time for regions vs nationally.

Visualization 2: Horizontal Bar Graph- How much did fertility drop? (% Change between 2010-2023)

```python
#Visualisation 2: Percent Change

# Calculating % change from 2010 to 2023 for each region and England
def get_pct_change(group):
    v2010 = group[group['Time period'] == 2010]['Value'].values
    v2023 = group[group['Time period'] == 2023]['Value'].values
    if len(v2010) > 0 and len(v2023) > 0:
        return ((v2023[0] - v2010[0]) / v2010[0]) * 100
    return None

reg_changes = df_Regions.groupby('Area Name').apply(get_pct_change).sort_values()
eng_change = get_pct_change(df_england_sum)

plt.figure(figsize=(10, 6))
colors = ['red' if x < eng_change else 'skyblue' for x in reg_changes]
reg_changes.plot(kind='barh', color=colors)
plt.axvline(eng_change, color='black', linestyle='--', label=f'England Average ({eng_change:.1f}%)')
plt.title('How much did fertility drop? (% Change 2010-2023)')
plt.xlabel('Percentage Change (%)')
plt.legend()
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Data Viz/iz2_fertility_drop.png')
```

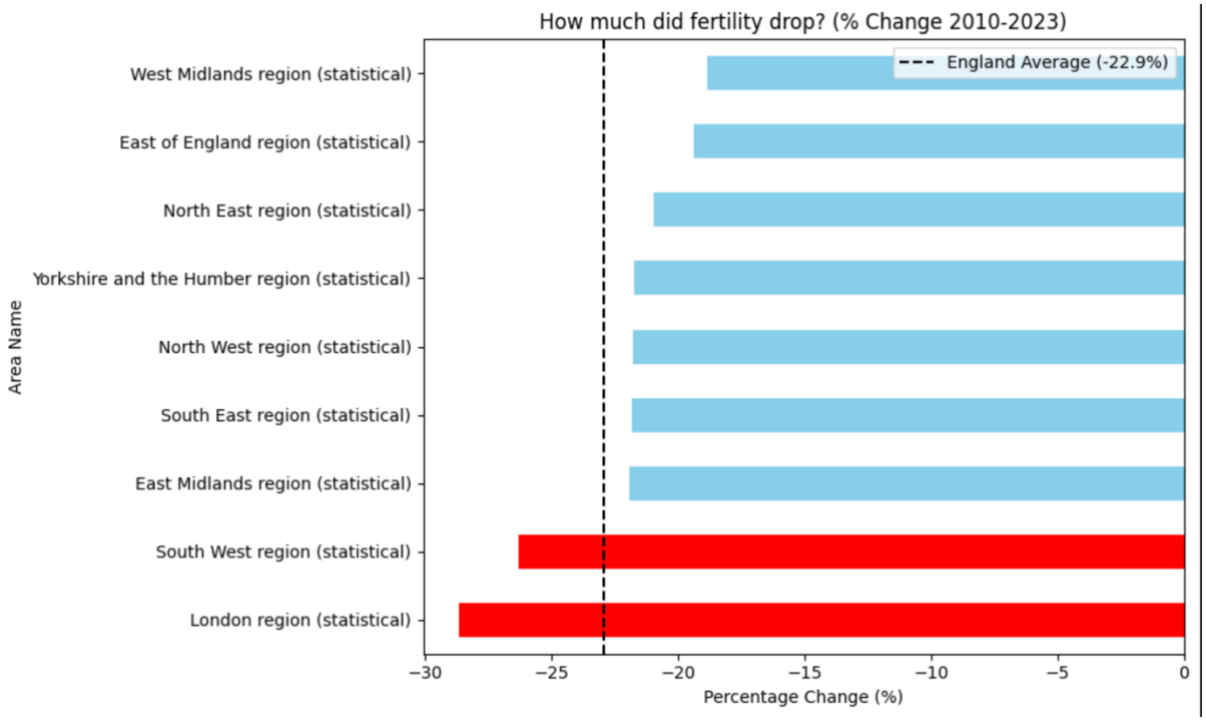

Figure 17: Visualization 2

I wanted to prove that while every region is falling, some are "crashing" faster than others.

Visualization 3: Heat Map- Regional Fertility Rates Year-by-Year

```
#Visualisation 3:Heatmap

pivot_reg = df_Regions.pivot(index='Area Name', columns='Time period', values='Value')
plt.figure(figsize=(12, 8))
sns.heatmap(pivot_reg, annot=True, fmt=".1f", cmap="YlOrRd")
plt.title('Regional Fertility Rates Year-by-Year')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Data Viz/viz3_heatmap.png')
```

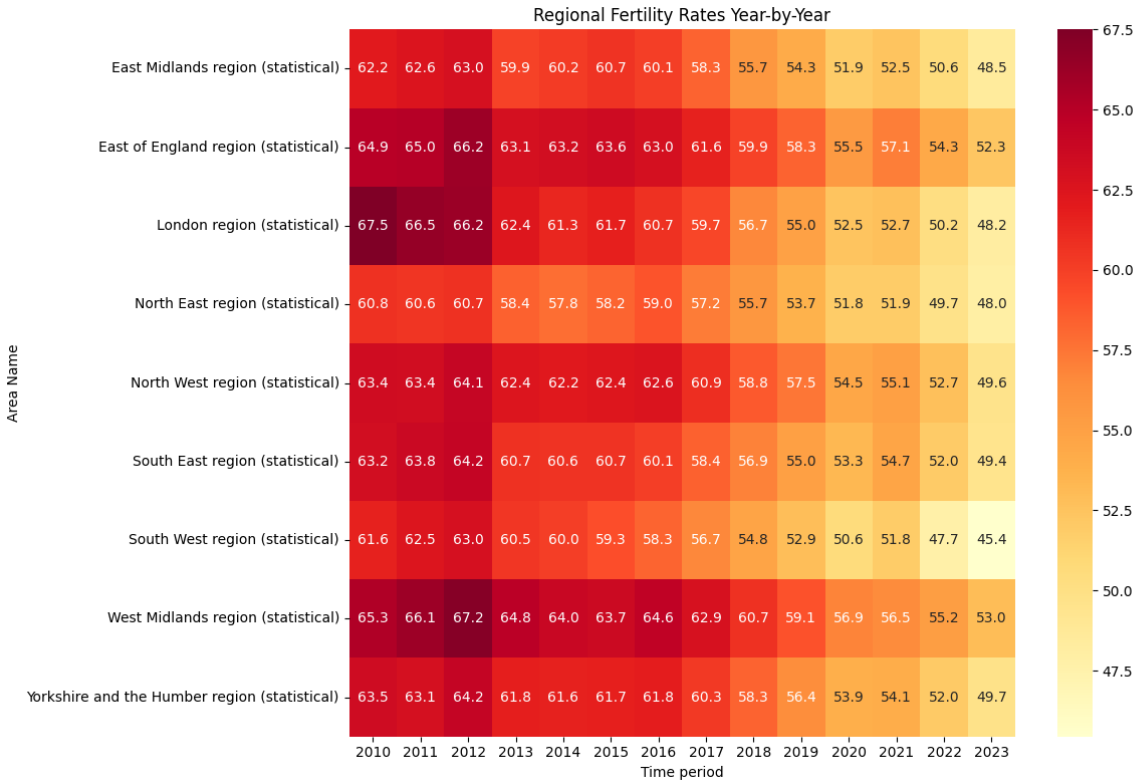

Figure 18: Visualization 3

The colour changes in heatmap makes it obvious that rankings of GFR each year almost never change.

Visualization 4: Side-by-Side Line Graphs- Regional Trends compared to National Average

```
# Visualisation 4: Side-by-Side Comparison

g = sns.FacetGrid(df_Regions, col="Area Name", col_wrap=3, height=3, aspect=1.5)
g.map(sns.lineplot, "Time period", "Value", color="blue", alpha=0.7)

# Overlay England trend in each
for ax in g.axes.flatten():
    sns.lineplot(data=df_england_sum, x='Time period', y='Value', color='black', ax=ax, linestyle='--', alpha=0.5)
g.set_titles("{col_name}")
plt.subplots_adjust(top=0.9)
g.fig.suptitle('Regional Trends (Blue) compared to National Average (Dashed Black)')
plt.savefig('/content/drive/MyDrive/Data Viz/viz4_facets.png')
```

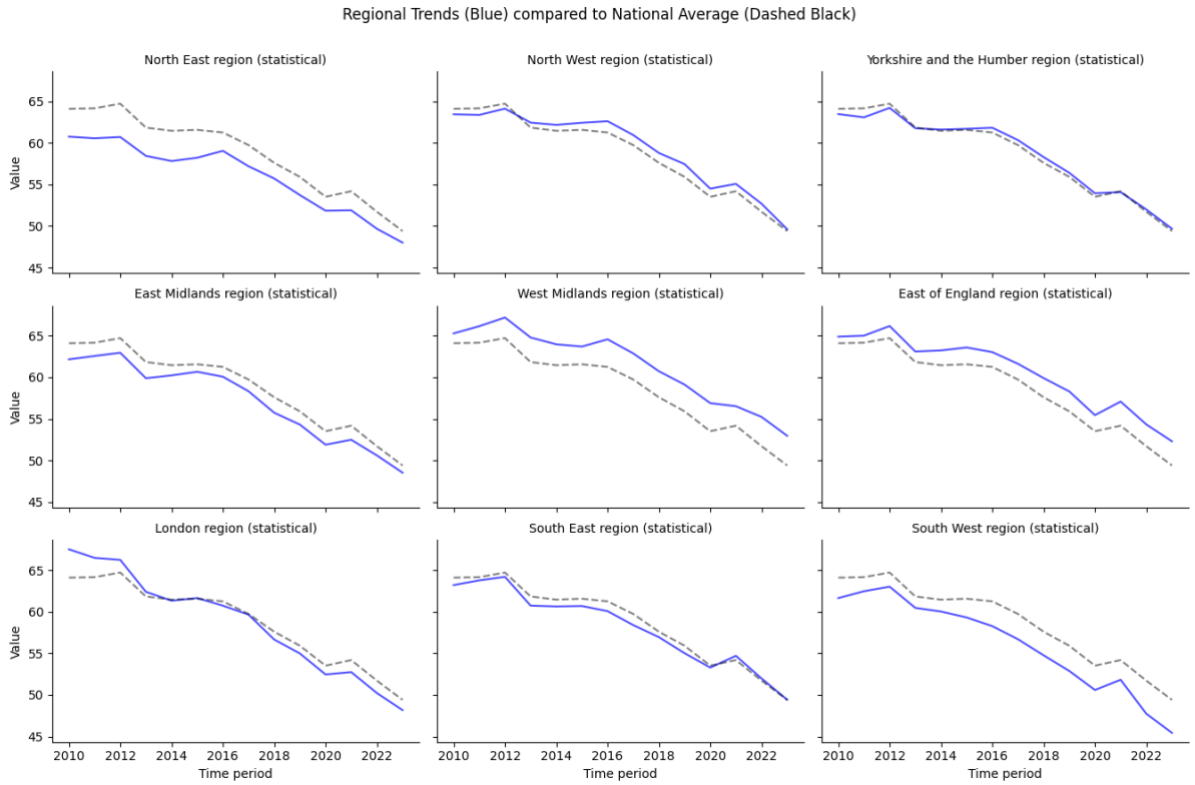

Figure 19: Visualization 4

Giving each region its own box, allows me to see how each region deviates from the England average.

Visualization 5: Box Plot- Variation of fertility rate within Local Districts within each Region (2023)

```
#Visualisation 5: Boxplot

plt.figure(figsize=(12, 6))

# Focus on the most recent year 2023
df_dist_2023 = df_districts[df_districts['Time period'] == 2023]
sns.boxplot(data=df_dist_2023, x='Value', y='Parent Name', palette='Set3')
plt.axvline(df_england_sum[df_england_sum['Time period'] == 2023]['Value'].values[0], color='red', label='England Average')
plt.title('Variation in Local Areas (Districts) within each Region (2023)')
plt.xlabel('Fertility Rate')
plt.legend()
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Data Viz/viz5_facets.png')
```

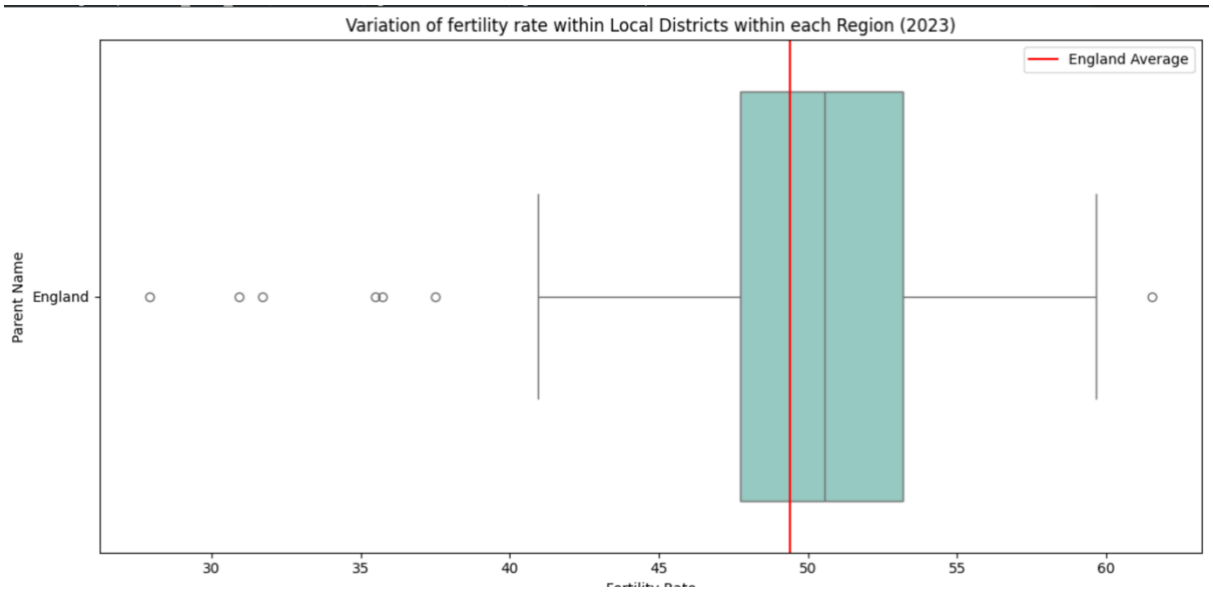

Figure 20: Visualization 5

Boxplot is the best univariate graph to highlight variation in data.

Visualization 6: Line Chart with Shaded Gap- National Fertility Rate compared to Highest vs. Lowest Local District GFR (2023)

```
#Visualisation 6

spread = df_districts.groupby('Time period')['Value'].agg(['min', 'max', 'mean']).reset_index()
plt.figure(figsize=(10, 6))
plt.fill_between(spread['Time period'], spread['min'], spread['max'], color='gray', alpha=0.2, label='Range between highest/lowest district')
plt.plot(spread['Time period'], spread['mean'], color='blue', label='District Average')
plt.plot(df_england_sum['Time period'], df_england_sum['Value'], color='black', linewidth=2, label='England National')
plt.title('The Gap: Highest vs. Lowest Local Districts over Time')
plt.ylabel('Fertility Rate')
plt.legend()
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Data Viz/viz6_spread_gap.png')
```

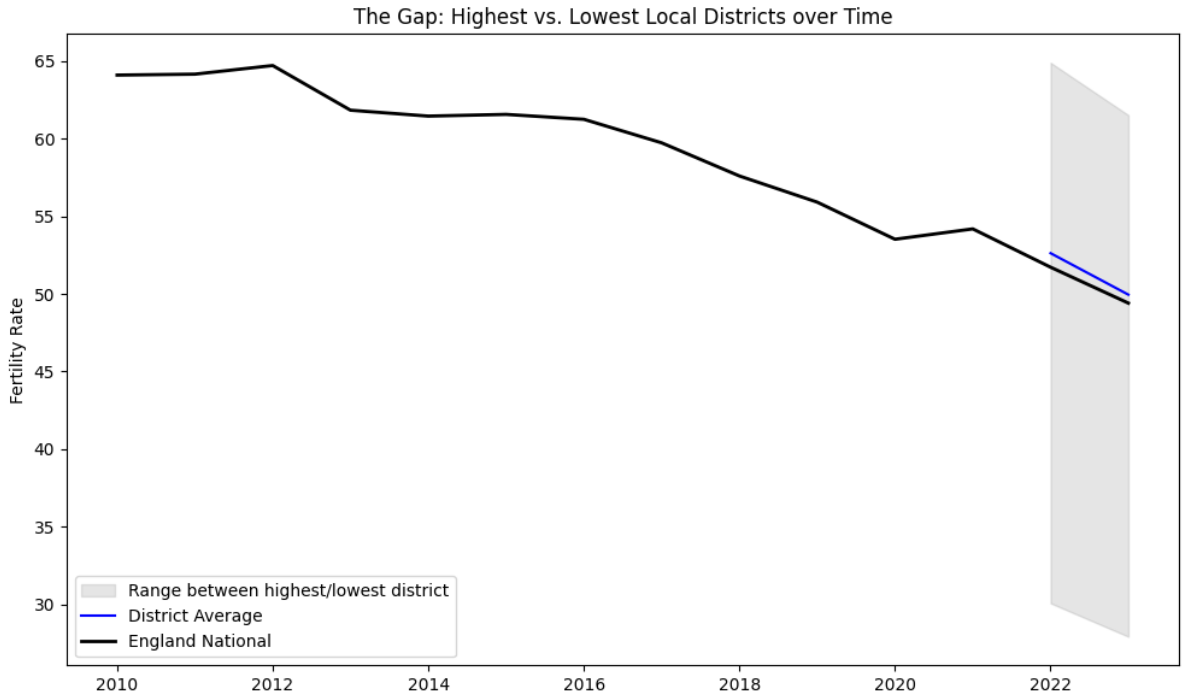

Figure 21: Visualization 6

I wanted to see if the GFR of the most and least fertile districts was getting closer to national average.

```python
# Final check of numbers for insights to include in the report

print("England's national fertility rate for 2010 and 2023")
print("England 2010:", df_england_sum[df_england_sum['Time period'] == 2010]['Value'].values)
print("England 2023:", df_england_sum[df_england_sum['Time period'] == 2023]['Value'].values)

print()

print("England's maximum and minimum district fertility rate for 2023")
print("Max District 2023:", df_dist_2023['Value'].max())
print("Min District 2023:", df_dist_2023['Value'].min())
```

```
England's national fertility rate for 2010 and 2023
England 2010: [64.09782852]
England 2023: [49.40598]

England's maximum and minimum district fertility rate for 2023
Max District 2023: 61.52156
Min District 2023: 27.91705
```

Figure 22: Key fertility rate values for comparison

How the research question was refined

Research question refined by moving from "is fertility falling?" to a more specific "how does the level of geographic aggregation change our perspective?" combining temporal and spatial analysis.

Use of Generative AI

Generative AI was helpful throughout this work. Gemini was interesting as it not just gives the ideas it’s a fast way to generate the visuals. Although the accuracy of these visuals are questionable, they did help me gain some basic sense of what my work is going to look like. This helped support early-stage planning.

In addition, I used the Gemini feature in Google Collab to debug and refine my code. It made fixing errors faster and also helped improve visualizations by adding features into them. This allowed me to spend more time interpreting results than wasting it on coding.

Overall, generative AI supported many steps of the work. It was like having someone I could discuss my work with. It saved time, reduced errors, and helped me present a clearer data story.